

Hiding data in Btrfs file systems

Fergus Toolan*, Georgina Humphries

Norwegian Police University College, Oslo, Norway

HIGHLIGHTS

- Data hiding techniques in Btrfs file systems.
- Novel hiding technique utilising STRING_ITEMS.
- Evaluation of Data Hiding techniques.

ARTICLE INFO

Keywords:

Data hiding
Anti-forensics
File systems
Btrfs

ABSTRACT

Data hiding in file systems is a common form of anti-forensics, however, most research in this area has been conducted on common file systems such as FAT, NTFS, Ext, etc., and not lesser used file systems. This research focuses on data hiding in Btrfs file systems. While Btrfs is now the standard for openSUSE and Fedora Linux distributions forensic tools still provide little support for the file system. This paper proposes six new techniques for data hiding in the Btrfs file system and evaluates their effectiveness using the metrics of capacity, stability and detection difficulty.

1. Introduction

File system forensics generally focuses on how information can be recovered from file systems (Carrier, 2005; Juch, 2014; Bhat and Wani, 2018; Toolan, 2025), rather than on how information can be hidden in these same file systems. This research is further lacking in the case of less common file systems such as Btrfs.

The Btrfs file system was initially created in 2007 and, in 2015 and 2020, openSUSE and Fedora adopted Btrfs as their default file systems respectively. Btrfs provides support for large-scale data storage solutions through features such as: in-built integrity checks, in-built volume management (i.e., RAID), snapshots and subvolumes, and SSD optimisation (Rodeh et al., 2013; Btrfs (nd), 2026a).

From both the forensic and anti-forensic viewpoints, little research has been conducted on the Btrfs file system. For instance, none of the commercial file system forensic tools support Btrfs. The Sleuthkit (Version 4.14) provides basic support for the Btrfs file system. A limited number of data hiding techniques have been implemented in the Btrfs file system (Göbel et al., 2024). The aim of this research is to determine what other data hiding techniques can be implemented in the Btrfs file system.

This paper is organised as follows. Section 2 outlines the related work. This section introduces the Btrfs file system structures and provides an introduction to anti-forensics with a focus on data hiding in file systems. Finally, this section introduces existing data hiding methods in Btrfs. Section 3 describes the six data hiding methods that are introduced in this research. Section 4 describes the evaluation methods employed in this study and presents the results of each implemented method. These results are then discussed in Section 5. Section 6 introduces the limitations of this study and proposes possible directions for future work. The paper concludes in Section 7.

2. Related work

This section first examines the Btrfs file system itself (Section 2.1) and describes its main components. Section 2.2 continues with an introduction to the area of anti-forensics with a specific focus on data hiding in file systems. To date, only a small number of data hiding techniques have been implemented in the Btrfs file system. Section 2.3 discusses previous efforts to hide data in the Btrfs file system.

* Corresponding author.

Email address: fertoo@phs.no (F. Toolan).

2.1. Btrfs file system

The Btrfs file system is a modern B-Tree based file system utilising copy-on-write (CoW) (Rodeh et al., 2013). It is currently used as standard in openSUSE (openSUSE, 2025) and Fedora (Fedora Project, 2020) Linux distributions. It is also fully supported in the Linux kernel and can therefore be encountered on any Linux distribution (Btrfs (nd), 2026c).

Btrfs allows for file systems that span multiple devices. This may be in the form of creating a larger volume from all components or implementing a Redundant Array of Independent Disks (RAID) solution through the file system (Toolan, 2025). Indeed, by default all Btrfs file systems implement a basic form of RAID level 1 as both the metadata and data chunks are mirrored (Rodeh et al., 2013).

One of the features of Btrfs that makes it more challenging for data hiding is that it utilises integrated checksums (Toolan, 2025). All tree nodes (and other file system structures) contain checksums to ensure their integrity. Changes to the underlying data (i.e., through data hiding) will alter these checksums and lead to detection. This is similar to the XFS file system in which checksums must be updated after hiding data (Toolan and Humphries, 2025a).

Btrfs can also include sub-volumes and snapshots. A sub-volume is a smaller volume inside the main Btrfs file system (Toolan, 2025). Sub-volumes are implemented as separate file system trees in the Btrfs file system (Rodeh et al., 2013). All of these tree structures are found from the tree of trees (also called the root tree). This structure contains information on all trees in the file system and is found in the superblock (Toolan, 2025).

Like many file systems Btrfs contains a superblock structure. Depending on the volume size there may be up to three superblocks on each file system device. The superblocks are located at offsets 0x10000 (64KiB), 0x4000000 (64MiB), and 0x4000000000 (256GiB). The superblock is used to recognise a Btrfs file system and to locate the main file system structures (Göbel et al., 2024).

As stated previously, Btrfs is a tree-based file system. The root tree structure maintains information on all of the trees present in the file system. The file system tree contains information about all files/directories in the volume. Each sub-volume will have its own file system tree (Toolan, 2025). However, there are other trees in the file system that might be relevant to certain data hiding methods presented in this paper (e.g., internal nodes - Section 3.5). These include:

- Extent Tree: This tree contains information about all allocated extents in the file system and is used to map free space in the file system (Rodeh et al., 2013).
- Extent CSUM Tree: This holds extent csum items for every allocated extent in the file system. The extent csum is a checksum calculated over each extent. This tree can be used to ensure data integrity (Btrfs (nd), 2026a).
- Device Tree: This tree contains information about each device in the file system.
- Chunk Tree: This allows the mapping of logical to physical addresses and, when combined with the device tree, allows for the implementation of RAID systems (Rodeh et al., 2013).

The Btrfs file system also contains many other trees such as the sub-volume tree, data relocation tree, orphan tree, etc. (Btrfs (nd), 2026a).

2.2. Anti-forensics & data hiding

Conlan et al. (2016) define anti-forensics as “attempts to alter, disrupt, negate, or in any way interfere with scientifically valid forensic investigations”. They divide anti-forensics into five distinct sub-types, expanding upon the four types originally defined by Rogers (2005). These types are data hiding (Conlan et al., 2016; Toolan and Humphries, 2025b), artifact wiping (Rogers, 2005), trail obfuscation (Conlan et al., 2016), attacks against the forensic process (Rogers, 2005) and indications of anti-forensics (Gül and Kugu, 2017; González Arias et al., 2024).

This paper focuses on data hiding in Btrfs file system structures. Göbel and Baier (2018) considers there to be three categories of file system data hiding:

- Hiding data in slack space: Slack space exists at the end of many file system structures (and files) that do not occupy the entirety of their final cluster/block. Data can be hidden in this slack space. There are many examples of this in past research including: file/directory slack space; superblock slack space (Berghel et al., 2008); block/inode bitmaps (Göbel and Baier, 2018); symbolic link slack space (Toolan and Humphries, 2025b); etc.
- Hiding data in reserved space: Many file systems contain reserved space both at the block level (e.g., the free list area in XFS (Toolan and Humphries, 2025a)) and internal to particular structures (e.g., in the ExFAT volume boot record (Toolan, 2025)). These reserved areas can be used to hide data.
- Misusing file system structures to hide data: It is also possible to misuse file system structures in order to hide data. This includes examples such as marking clusters as allocated and using these clusters to hide data (Berghel et al., 2008) and using nanosecond timestamp components (Göbel and Baier, 2018).

2.3. Data hiding in Btrfs

To date little focus has been given to data hiding techniques on the Btrfs file system. Wani et al. (2020) attempted a number of anti-forensic techniques on Btrfs, including some data hiding techniques, with limited success. They attempted to use the 0x10000 bytes of space prior to the first superblock as a means of hiding data, along with reserved areas in the root item and the free space in both leaf and internal nodes. Wani et al. (2020) noted that data hidden in the superblock reserved areas was overwritten upon mounting. The other methods of data hiding attempted by Wani et al. (2020) failed due to the in-built integrity checking in Btrfs.

Göbel et al. (2024) have successfully implemented two data hiding techniques for Btrfs as part of the fishy framework (Göbel and Baier, 2019) for generating data sets with anti-forensic traces. These are two traditional hiding methods that have been implemented in many file systems (Berghel et al., 2008; Göbel and Baier, 2018; Toolan and Humphries, 2025a):

- File Slack Hiding: Extents in Btrfs are allocated at the block level making it possible for slack space to exist. This technique gathers the EXTENT_DATA and INODE_ITEM items for the particular file and calculates the available slack space from these structures. Data can be hidden in this slack space. The implementation of this technique requires two additional updates to the file system. The checksum tree contains a checksum item for each extent in the file system (Rodeh et al., 2013). The checksum item contains a hash value of the extent which must be updated after data is hidden in the extent's slack space. Once this update has occurred the checksum tree node header's checksum is now incorrect and must also be updated (Göbel et al., 2024).
- Nanosecond Timestamp Hiding: Another common data hiding technique implemented on Btrfs by Göbel et al. (2024) uses the nanosecond component of the timestamps to hide information. Each inode in Btrfs contains four four-byte nanosecond values (modified, accessed, changed and created) allowing for 16_d bytes of storage per inode. Once the inode is updated the node header's checksum also requires updating.

Göbel et al. (2024) propose a number of other possible techniques for data hiding in Btrfs. Firstly, they propose using the unallocated 64MiB before each of the superblock structures. This space is meant to be zeroed and is not checked by the file system drivers. Hence, it would make an ideal hiding location.

Secondly Göbel et al. (2024) also proposes the use of tree node slack space for data hiding. Internal nodes generally contain slack space and as such would prove to be likely candidates for effective data hiding.

This would be particularly true for large file systems, based either on the number of files in the file system or the number of devices whereby redundancy of nodes would allow for much greater hiding capacities.

Schwietert and Hilgert (2025) proposed using the reserved areas in the superblock for data hiding purposes. There are between 1 and 3 copies of the superblock on each device in a Btrfs file system. Each superblock contains 240_d bytes of reserved space and 565_d bytes of slack space (Toolan, 2025; Schwietert and Hilgert, 2025). As with the other implemented data hiding techniques in Btrfs the superblock checksums would also need to be updated if this hiding strategy is employed.

3. Methods

We introduce six methods of data hiding for the Btrfs file system. A number of these methods were proposed, but not implemented, by other authors such as using the superblock's reserved and slack space (Schwietert and Hilgert, 2025), internal tree node slack space (Göbel et al., 2024) and use of the 64KiB space before each superblock (Göbel et al., 2024). Other techniques implemented in this paper include using the reserved space in the inodes, the slack space of the superblock's chunk array and finally the use of a Btrfs STRING_ITEM structure for data hiding. This section describes each of the above methods of data hiding and how they can be realised in Btrfs file systems.

One difficulty in implementing data hiding in the Btrfs file system is the use of integrity checking in the file system. By default the crc32c checksum algorithm is used (Btrfs (nd), 2026b), however, the file system can use a variety of algorithms including xxhash, sha256 and blake2. Almost all metadata in Btrfs is found in trees. Each tree node contains a checksum value meaning any changes to metadata will invalidate the file system unless the checksum is also updated. Another primary source of file system information is the superblock which also has a checksum (Btrfs (nd), 2026a). A number of the techniques proposed in this section will update these structures and will also require checksum values to be updated when data is hidden. In the case of both tree nodes and the superblock, the checksum is located in the first 0x20 bytes of the structure. The checksum is calculated over the entire block less the first 0x20 checksum bytes. Additionally, the checksum tree maintains a series of extent csum items which record the checksum of all extents in the file system. This means that even the simple exploitation of file slack requires checksum updating in the Btrfs file system (Göbel et al., 2024).

3.1. Pre-superblock

Btrfs file systems contain up to three copies of the superblock on each device present. The first of these is always found at byte offset 0x10000 (64KiB).¹ In Btrfs file systems the bytes prior to the first superblock are always zero. This area provides 0x10000 bytes of possible hiding space (Wani et al., 2020).

Göbel et al. (2024) claim that there exist 64KiB bytes of free space prior to all superblocks in the file system, however, this does not appear to be the case. Fig. 1 shows the contents of the node immediately before the secondary superblock on a Btrfs file system. This node clearly contains data related to the file system in question (UUIDs are the same). Hence, it would appear that this method is only feasible prior to the first superblock.

This pre-superblock area is not checksummed and as such no further action should be required when hiding data using this method.

3.2. Superblock reserved/Slack space

Both Schwietert and Hilgert (2025) and Göbel et al. (2024) mention the possibility of hiding data in the reserved and slack areas of the superblocks. This has been attempted by Wani et al. (2020) but it resulted in the file system becoming unmountable. This appears to be due

to checksum integrity issues. This, modified technique, not only hides data in the reserved/slack area of the superblock, but also updates the checksum.

There are two fixed size areas in the superblock, one reserved area and a slack space area. These are:

- Reserved Area: 0xF0 bytes at 0x23B
- Slack Space: 0x235 bytes at offset 0xDCB (Toolan, 2025)

Data can be written directly to these areas allowing for 0x325 bytes of storage per superblock structure. The final step when wishing to hide data in the superblock requires the superblock checksum value to be updated to reflect the changes to the superblock content and ensure file system consistency.

3.3. Chunk array slack space

In addition to these fixed-length reserved and slack areas in the superblock there is also potentially more free space available. Btrfs uses logical addresses to locate objects in the file system. These logical addresses are then mapped to physical addresses through the chunk tree (Rodeh et al., 2013; Bhat and Wani, 2018; Toolan, 2025). However, in order to locate the chunk tree it is necessary to map the chunk tree's logical address to a physical device/offset. To accomplish this, a small part of the chunk tree, called the chunk array, is stored in the Btrfs superblock.

Btrfs allocates 0x800 bytes in the superblock for storage of the chunk array, however, the chunk array rarely occupies all of this size. Generally, in a single device file system the chunk array occupies 0x81 bytes, leaving 0x77F bytes for data hiding. As with the previous method (Section 3.2) implementing the chunk array slack space data hiding method requires the superblock checksum to be updated.

3.4. INODE_ITEM reserved space

Schwietert and Hilgert (2025) propose the use of reserved space in the Btrfs inode structure as a means of data hiding. Btrfs inodes are found in INODE_ITEMS in the file system tree structure (Rodeh et al., 2013; Bhat and Wani, 2018; Toolan, 2025). There exists a single INODE_ITEM for each file in the file system. Each INODE_ITEM contains 0x20 bytes of reserved space at offset 0x50. This space can be used for data hiding purposes.

INODE_ITEMS exist in the file system tree structures. As they exist in tree nodes they are also subject to integrity checks. The node header of each Btrfs tree node contains the checksum for all data in the node (except the first 0x20 bytes in which the checksum value itself is stored). After injecting data into the INODE_ITEM's reserved area the tree node's checksum must be updated to maintain integrity.

3.5. Internal nodes

Göbel et al. (2024) describe the possibilities of using tree nodes, in particular internal nodes, to hide data. As with many modern file systems Btrfs uses tree structures to store much information. Trees contain both leaf and internal (or index) nodes. Leaf nodes consist of a header, immediately followed by data pointers. These point to the actual items in the node which are found at the end of the node. The leaf node contains free space between the end of the data pointer area and the value area. This free space will shrink, both when the data pointer area expands and when the value area expands. The leaf node structure is shown in Fig. 2(A).

The internal node structure consists of the node header followed by pointers to other nodes. The remainder of the node consists of free space. Unlike leaf nodes growth only occurs when new pointers are added to the node structure. This action does not occur with such frequency as the shrinking of leaf node free space, making this a potentially robust method of data hiding (Göbel et al., 2024). The internal node structure is shown in Fig. 2(B).

¹ The others are found at 0x4000000 (64 MiB) and 0x4000000000 (256 GiB) offsets if present.

03ffc000:	5214	cff1	0000	0000	0000	0000	0000	0000	R.....
03ffc010:	0000	0000	0000	0000	0000	0000	0000	0000	
03ffc020:	4eef	616b	c932	4249	a2e6	f8a0	5ded	76f4	N.ak.2BI....].v.
03ffc030:	00c0	7f03	0000	0000	0100	0000	0000	0001	
03ffc040:	2302	89c9	735f	4575	928d	391b	8f5b	80f8	#...s_Eu..9..[..
03ffc050:	dd01	0000	0000	0000	faff	ffff	ffff	ffff	
03ffc060:	0100	0000	00fa	ffff	ffff	ffff	ff84	0101	
03ffc070:	0000	0000	0000	e43d	0000	b701	0000	0000	=.....
<snip>									
04000000:	f16b	bb10	0000	0000	0000	0000	0000	0000	.k.....
04000010:	0000	0000	0000	0000	0000	0000	0000	0000	
04000020:	4eef	616b	c932	4249	a2e6	f8a0	5ded	76f4	N.ak.2BI....].v.
04000030:	0000	0004	0000	0000	0100	0000	0000	0000	
04000040:	5f42	4852	6653	5f4d	6c03	0000	0000	0000	_BHRfS_M1.....
04000050:	00c0	5660	0200	0000	0080	d02b	0300	0000	..V'.....+....

Fig. 1. Excerpt from a Btrfs file system showing a valid file system node 0x4000 bytes prior to the second superblock.

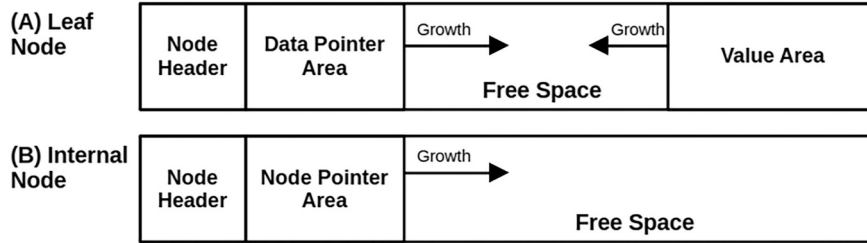


Fig. 2. Node structure in Btrfs B-Trees. The topmost node (A) is a leaf node and the other (B) is an internal node.

This method involves locating all internal nodes in the file system as potential targets for data hiding. Once located, information can be written to the end of the node. Generally, it is better to leave space between the end of the data pointer area and the injected information. This will ensure that B-Tree growth is less likely to overwrite the contents of the internal node's slack space. As with all data hiding techniques that alter the contents of a B-Tree node, the node's checksum must be updated to reflect the hidden data in order to avoid detection of this technique.

3.6. STRING_ITEM

Tree nodes in Btrfs contain many different item types. The Btrfs file system defines a STRING_ITEM type which can be created in any tree. According to the On-disk Format page in the Btrfs documentation (Btrfs nd, 2026a) this item contains a string and is currently 'used only for testing'. The idea behind this method is to use STRING_ITEMS for data hiding purposes. To do this requires performing the following steps:

1. Locate a leaf node in any file system tree with free space between the key and value areas.
2. Get the number of items in the node ($N_{original}$) from the node header. This is found at offset 0x60 in the header (Toolan, 2025).
3. Increase the number of items in the node header by one (i.e., $N_{original} + 1$) and write this value to the node.
4. Create a STRING_ITEM (Type: 0xFD) key value. As with all key structures this will be a 17_d byte structure with an 8_d byte OID, followed by an item type (0xFD - for a STRING_ITEM) and a subsequent 8_d byte key offset value (Rodeh et al., 2013). This value can be given as 0x00 while the OID should have the value of the largest OID in the current node. This will mean that it will be sorted correctly when inserted at the end of the node.
5. Get the offset to the start of the value area. This is achieved by checking all existing keys and locating the smallest offset value

(O_{min}). The final offset is given as $O_{min} - M_{length}$ where M_{length} is the length of the message to be hidden.

6. The final data item pointer structure is given as the key (Step 4) followed by the data offset ($O_{min} - M_{length}$) followed by the data item size (M_{length}). This is written to the image at the end of the data item pointer area. As we are only utilising leaf nodes each data pointer is 25_d bytes in size, a 17_d byte key and two four byte values for the offset and size of the item (Rodeh et al., 2013; Toolan, 2025). Hence, the end of the data pointer area is located at offset $0x65 + (N_{original} \times 25)$ in the node, where 0x65 is the header size in bytes.
7. The message is then written to offset $O_{min} - M_{length}$.
8. As with using the INODE_ITEM's reserved space (Section 3.4) the node header's checksum value must now be updated to reflect the changes made in the file system tree.

This is an example of misusing file system structures to hide data. The STRING_ITEM is generally never found in a live Btrfs file system, but it is a valid file system structure and as such should not cause any issues to the functionality of the file system.

4. Evaluation

Göbel and Baier (2019), building upon their previous work (Göbel and Baier, 2018), proposed the use of three metrics to evaluate data hiding techniques: capacity, detection rating, and stability. Capacity measures the amount of data that can be hidden using a particular technique. In order to create an effective data hiding technique its capacity must be sufficiently large to cope with large volumes of data. The detection rating evaluates data hiding techniques based on how likely a forensic analyst is to detect the hidden information. This is rated as easy, medium and hard. Finally, stability measures the persistence of the hidden data in the file system through normal usage. For a data hiding

technique to be effective it must protect the hidden information from being overwritten through file system usage. This study uses the capacity and stability metrics as used in Göbel and Baier (2019) and adopts the detection difficulty metric in place of the detection rating as used by Toolan and Humphries (2025a). All three metrics are rated as low, medium and high.

In a manner similar to that employed by Toolan and Humphries (2025a) detection difficulty is considered through three use cases. These are: the user level; the forensic analyst level; and the system administrator level. It is necessary to evaluate how different types of interaction with the file system will affect the detection probability.

For each data hiding technique a disk image was prepared. This device was zeroed initially and then a Btrfs file system was created on the device. A number of files/directories were created on this device. This image was used as the basis for evaluation for all methods presented in Section 3 except the internal node method. In order to evaluate this technique it was necessary to have a file system that contained internal nodes. The original file system was modified by adding 1000 files to it. This ensures that the file system tree will contain internal nodes.

Prior to data hiding all of the tests were run on the two original images and the results of these were saved. The tests include:

- User Level
 - Mounting the file system
 - File creation
 - File modification
 - File deletion
 - Unmounting the file system
 - Note: The file creation/deletion actions result in changes to the file system tree. This in turn forces CoW. This then allows message recovery to be tested in a live file system.
- Forensic Analyst Level
 - Sleuthkit tests involve gathering file system information, listing all files in the file system, recovering metadata and file content. Version 4.14 of the Sleuthkit provides support for Btrfs.
- System Administrator Level
 - The **btrfs check** command is used to check the file system for errors.
 - The **btrfs check** with the **–check-data-csum** option is used to determine whether the data checksums are intact.
 - The **dmesg** command is executed during user interaction to ensure that no errors appear in the kernel logs.

Finally, after all interactions with the file system the message is recovered to ensure that the technique is stable in the case of normal file system usage. Table 1 shows the results of these tests. In the majority of cases, every test was passed by all techniques.

The exception is the internal node hiding mechanism. As the tree is updated nodes are altered using CoW. This has the effect of creating a

new copy of the node which is then modified. The original node is untouched. The new copy is created in such a way that the slack space is always zero'd. This means the hidden data is lost. The data will temporarily exist in the original tree node's slack space, however, this node will quickly become unallocated and as such the message is lost.

Btrfs file systems were created using two versions of **btrfs-progs** and tested with both versions. No differences were identified between the versions. All data hiding techniques appeared to work in an identical manner. Btrfs file system versions 6.6.3 and 6.19.1 were used on Linux Mint 22.3 and openSUSE 15.6 respectively.

The remainder of this section describes the evaluation of each individual technique in further detail.

4.1. Pre-superblock

The pre-superblock reserved area is 64KiB in size. Each device in a Btrfs file system has this area. Hence, the total capacity of this data hiding method is $(64 \times N)$ KiB where N is the total number of devices in the file system. While 64KiB in a single chunk is a large space for data hiding, the limited number of occurrences of these 64KiB chunks means that the overall capacity of this technique is medium.

In terms of stability, after all tests the hidden data was still present and unaltered. This technique should be highly stable, as the area used for data hiding is never utilised by the file system, and would appear to never be checked by the file system.

While the forensic analyst/system administrator levels of testing did not discover the information hidden in the pre-superblock space, we still consider this technique to have a low detection difficulty. This is due to the location of the hidden data. If a forensic analyst opens the image in a hex viewer they will immediately see the non-zero data in a reserved (zero'd) area. Any forensic analyst familiar with the structure of Btrfs should immediately detect this information.

4.2. Superblock reserved/Slack space

The combination of the reserved and slack space in the superblock provides a total of 805_d (0x325) bytes of storage space per superblock. This would lead to a maximum capacity of $805 \times 3 = 2415_d$ bytes per device. This assumes that the device is large enough to have three superblock structures. In the case of volumes smaller than 256GiB there will only be two superblock structures present. Consequently, the capacity of the superblock reserved/slack space is evaluated as low.

After testing, all data were still accessible in the superblock reserved and slack spaces. Hence, we rate the stability of this method as high.

This result contrasts findings by Wani et al. (2020) who state 'the hidden data are lost in subsequent re-mounts'. From this paper it would appear that the checksums for the superblocks were never updated in this case. This would cause errors during the mounting process.

This was tested by creating a new variant of the superblock data hiding technique that wrote data to the reserved/slack areas but failed to update the checksum value. Attempting to mount these resulted in errors, however, upon further analysis the hidden data was still present. Another possible explanation for this discrepancy is through the use of different versions of **btrfs-progs**. While Wani et al. (2020) does not state the exact version of the Btrfs file system used in their evaluation we can infer, based on the publication date and operating systems mentioned, that btrfs-progs 4.X or 5.X was most likely being used.

None of the system administrator or forensic analyst tests detected the hidden data. However, similarly to the previous attack, it is feasible that important structures such as the superblock may be studied at a lower level by forensic analysts. During a hex level analysis the hidden data is reasonably easy to detect, although not as easy as the pre-superblock data. For this reason we rate the detection difficulty as medium.

Table 1
Evaluation results for the various data hiding methods.

	Pre-Superblock Reserved	Superblock Reserved / Unused	Chunk Array Slack	INODE_ ITEM	STRING_ ITEM Misuse	Internal Node Slack Space
Mounting	Pass	Pass	Pass	Pass	Pass	Pass
Unmounting	Pass	Pass	Pass	Pass	Pass	Pass
User Actions	Pass	Pass	Pass	Pass	Pass	Pass
fsstat	Pass	Pass	Pass	Pass	Pass	Pass
fls	Pass	Pass	Pass	Pass	Pass	Pass
istat	Pass	Pass	Pass	Pass	Pass	Pass
icat	Pass	Pass	Pass	Pass	Pass	Pass
btrfs check	Pass	Pass	Pass	Pass	Pass	Pass
–check-data-csum	Pass	Pass	Pass	Pass	Pass	Pass
dmesg	Pass	Pass	Pass	Pass	Pass	Pass
Message Recovery	Pass	Pass	Pass	Pass	Pass	Fail

4.3. Chunk array slack space

As with the superblock reserved and slack space methods, the chunk array slack space's capacity is limited by the number of superblocks in the file system. Assuming the default chunk array size of 0x81 bytes the capacity of this method is 1919_d (0x77F) bytes of storage per superblock. However, it is unclear how often the chunk array structure grows during usage. Basic testing revealed that all created file systems have a chunk array size of 0x81 bytes. This involved creating file systems on larger devices (up to 50GiB) and involving up to four devices in various configurations (JBOD, Duplicated, RAID1, and RAID10). In all cases the chunk array size remained at 0x81. However, the limited number of superblocks still means the capacity of this data hiding method is low.

The detection difficulty of this method is identical to that of the superblock reserved/slack space areas and is considered to be medium. As the chunk array could theoretically grow during the file system's lifecycle we consider the stability of this method to be medium.

4.4. INODE_ITEM reserved space

The capacity of the reserved space in an INODE_ITEM is relatively low at a mere 0x20 bytes per INODE_ITEM. However, in file systems containing many files this can result in vast quantities of space for hiding information. Consider instances of Fedora and openSUSE, both of which use Btrfs by default.

The Fedora WS 43 instance contained 458,330 files/directories, while the openSUSE instance contained a total of 2,794,258² files. In the case of the Fedora workstation this would allow for approximately 14MB available for data hiding purposes using this technique. Hence, the capacity of this technique is rated as high.

Once the node checksum is successfully updated the chances of detection are slight. None of the forensic/system administration tools detect information hidden in the inode reserved area. However, we rate the overall detection difficulty as medium. As the INODE_ITEM is one of the most vital structures in file system forensics, it is more feasible that forensic analysts may look at the raw data for this structure. This is even more probable due to the lack of forensic tool support for the Btrfs file system. An analyst, with knowledge of file system forensics, should be able to identify more than mere zeros in the reserved area during analysis of the raw data.

The stability of this method of data hiding is high. The area that is used is reserved and therefore, without major changes to the file system structure, the data should never be altered/deleted.

4.5. Internal nodes

Göbel et al. (2024) proposed using the internal node slack space to hide data in Btrfs. They suggested that internal nodes often only utilise a small proportion of the available space. However, we found that this was not the case on file systems with a large number of files. Table 2 shows the available space for Fedora WS 43 and openSUSE 15.6 installations.

Generally, the capacity of this method is rated as medium. In large-scale file systems, while the available space in each internal node is low, there will be many nodes available. This means that the overall total is between 14Kb and 100Kb for the two sample file systems in Table 2. However, it should be noted that in the case of small file systems there may be a very small number of leaf nodes in the tree which would result in only a small amount of space being used in the internal nodes. This would then result in much storage capacity in one single node. The fact that only one node is available means that the overall capacity is small. However, as the file system grows the trees are balanced in such a way that there is very little slack space left in the internal nodes. Table 2

Table 2

Storage space available in internal nodes in Fedora and openSUSE.

	Fedora	openSUSE
Num. Files	458,330	2,794,258
Num. Internal Nodes	57	463
Num. Bytes	14,409	100,840
Avg. per Node	252.8	217.9

Table 3

Summary of the effectiveness of Btrfs data hiding techniques.

	Capacity	Detection Difficulty	Stability
Pre-Superblock	Medium	Low	High
Superblock	Low	Medium	High
Chunk Array	Low	Medium	Medium
INODE_ITEM	High	Medium	High
STRING_ITEM	High	High	High
Internal Node	Medium	High	Low

demonstrates that there are approximately 250 bytes per internal node in Fedora and 220 bytes in openSUSE.

Detecting information hidden in this method would be highly difficult. The system administration/forensic tools do not detect any tampering with the internal node structures. Furthermore, a forensic analyst is unlikely to examine internal nodes too closely. Potentially relevant case information will likely be found in the leaf nodes of the tree, and not in the internal nodes.

The problem with this technique lies in its stability. The message is only recoverable while the initial node remains available. Not only might internal node slack space be overwritten due to file system growth, but CoW also introduces difficulties for this technique. It appears that the internal node slack is zero'd every time the node is modified and a new instance is created due to CoW. This means that the hidden data only persists as long as the original altered node remains present. This will quickly become unallocated and as such is highly likely to be overwritten. Consequently, the hidden data becomes non-recoverable using this technique.

4.6. STRING_ITEM

The STRING_ITEM in Btrfs can be found in any leaf node of any tree and can be of any size, up to a maximum of $NodeSize - 0x65 - 0x19$ due to the node header (0x65 bytes) and the STRING_ITEM's key pointer (0x19 bytes). There can exist as many STRING_ITEMS in the file system as desired. In theory the capacity is only limited by the available free disk space. Hence, the capacity of STRING_ITEMS for data hiding is high.

The detection difficulty of these is also high. As the STRING_ITEM is only intended for debugging purposes the system administration tools appear unable to detect the item. Similarly, file system forensic tools are also unable to detect STRING_ITEMS. It would require a listing of all items in the tree node in order to potentially be recognised. This is unlikely, both at the forensic and system administration levels.

As with the detection difficulty the unexpected nature of the STRING_ITEM in a Btrfs file system means that it is a very stable method of hiding information in the file system. Hence, the stability is rated as high also.

4.7. Evaluation summary

This section evaluates the performance of the various proposed data hiding methods. The summary of this evaluation is presented in Table 3 and Fig. 3.

² The large discrepancy is due to Snapper's backup features being active in openSUSE.

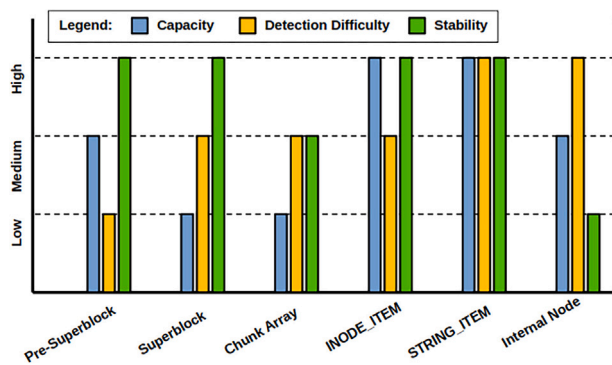


Fig. 3. Summary of the evaluation results of each technique in terms of capacity, detection difficulty and stability.

Referring to these, it is clear that the INODE_ITEM and STRING_ITEM methods are best for hiding data in the Btrfs file system. The INODE_ITEM method allows for much data to be hidden in large file systems. As this method uses reserved space it is highly stable. Indeed it should never be possible to overwrite the information. The INODE_ITEM structure is of such importance in file system forensics that it is possible that the analyst will manually examine the inode structure. For this reason, a particular issue with the INODE_ITEM method is that there is an increased chance of detecting the hidden information.

To alleviate this issue, the STRING_ITEM method was proposed. This uses a debugging feature of the Btrfs file system, that of a STRING_ITEM. This item can appear in any tree and can hold any desired information. As this item is not meant to appear in deployed file systems it is not reported by any of the system administration or forensic tools. This makes it very difficult to detect. As it is a valid item in the tree node, it is protected by the file system itself and as such is highly stable. Finally we can create as many of these items as we wish, meaning the technique has potentially vast storage capacity.

5. Discussion

Similar to XFS (Toolan and Humphries, 2025a), Btrfs is a modern file system that leads to more complexities in terms of hiding data in the file system. Older file systems (NTFS, FAT, etc.) had no integrity checking present, as such we could alter any structure that we wished with little chance of detection (once we ensured that we did not damage file system structures). Modern file systems, including Btrfs, have integrity checking mechanisms in many, if not all, structures. This means that, not only do we have to hide data, but we must also update the checksum value to ensure the integrity of the file system is maintained.

Table 3 provides a summary of the evaluation of the various data hiding techniques. Those techniques that involve the superblock structure itself are considered relatively ineffective for data hiding. The main reason for this is the limited number of superblocks. With only a maximum of three superblocks per device the maximum storage capacity of the superblock reserved/slack space technique is approximately 2.4KB per device. However, this level is only achievable in devices of 256GiB or larger sizes. While using the superblock chunk array's slack space results in larger storage capacity (allowing for a maximum of approximately 5.7KB per device) it is also limited in capacity due to the limited number of superblocks. It is also less stable than using reserved/slack space as the chunk array might grow in future versions of the file system. Additionally, all superblock related techniques are generally easier for forensic analysts to detect. The reason is that the superblock is such a vital structure in any file system that it is more likely to be analysed in detail, and possibly at the raw hex level than many other structures would be.

The use of the 64KiB of reserved space before the superblock is slightly more effective as a data hiding technique in that the capacity is

higher than the other superblock related techniques. However, as this occupies the first 0x10000 bytes on disk, the simple act of opening the disk image in a hex viewer should lead to the detection of this hidden data. This ease of detection means that the technique is not fully effective.

Some authors considered the use of internal nodes in B-Trees to be a worthwhile method of data hiding (Göbel et al., 2024; Schwieter and Hilgert, 2025). However the method appears to not be as effective as thought. This is due to the means by which Btrfs implements Copy-on-Write (CoW). When a copy is made of a node the slack space is not copied. This means that slack space is overwritten by zeros at each copy. While the hidden data remains in the original node, this node will quickly become unallocated. This means that the hidden data is no longer protected by the file system and can then be overwritten.

The INODE_ITEM method provides the potential for large storage capacities. The capacity is related to the number of files (and therefore inodes) present in the file system. While only 32_d bytes are available per inode, the use of multiple inodes allows for potentially large volumes of storage. In the case of the Fedora and openSUSE file systems in Table 2 this technique allows for approximately 14MB and 89MB respectively.

Using a STRING_ITEM is a novel means of hiding information in the Btrfs file system. This technique is an example of misusing file system structures, one of the common data hiding categories. However, it is slightly different in that the STRING_ITEM should not appear in a live file system. It is only to be used for testing purposes (Btrfs (nd), 2026a). The running file system (and forensic tools) ignores the structure completely. As such, it is very difficult for anyone to detect this method. As many STRING_ITEMS can be created as desired. Each can be created to any size up to the limits of node size. This means the potential for data storage is immense. Overall, the use of STRING_ITEMS appears to be the most effective data hiding method in Btrfs evaluated in this research.

6. Limitations & future work

This section examines the limitations of the presented work and suggests some means of continuing this work in the future. There are two main limitations of the current work:

- By default all Btrfs file systems use the crc32c checksum algorithm. However, they can also support the xxhash, sha256 and blake2 hashing algorithms (Btrfs (nd), 2026b). This research has only focused on the crc32c algorithm. While it is expected that the choice of checksum algorithm should not make any difference to the data hiding techniques, this should be confirmed through testing.
- All file systems used in this research were created using mkfs.btrfs with default options. It should be investigated whether other options might change the effectiveness of the data hiding techniques evaluated in this research.

There are a number of potential directions for future work in this area. Firstly, the above limitations could be addressed, which would include testing these techniques using checksums other than crc32c and testing on file systems that are created using alternative options.

Another direction for further research is to build a toolkit to detect data hiding in the Btrfs file system, and also in other file systems. Many of the evaluated techniques would be very easy to detect, if there were a tool to do so. For instance, the INODE_ITEM, pre-superblock, and superblock methods use reserved space which should be zero. Similarly, the chunk array and internal nodes use slack space which should be easy to detect. Even the STRING_ITEM method should be possible to detect as these items should not exist in the file system.

A common data hiding technique is to manipulate the file system's allocation bitmap structures or bad block structures to mark blocks as being used meaning the file system will not try to overwrite them. These marked blocks can then be used for data hiding (Göbel and Baier, 2019; Toolan and Humphries, 2025a). Allocated blocks are identified in a different manner in the Btrfs file system through the use of extents. It

might be possible to create extents in this extent tree and then use the 'allocated' blocks to hide data.

7. Conclusions

This paper provides a discussion of data hiding techniques in the Btrfs file system. While Btrfs is a lesser used Linux/Unix based file system, it is now the default in Fedora and openSUSE distributions and will occasionally be encountered during investigations. Currently, there is little forensic tool support for Btrfs making it potentially easier and more effective to hide information in the system due to the investigator's lack of knowledge and tools.

This paper investigates six means of hiding data in the file system and shows how effective each might be. This is evaluated under the auspices of capacity, detection difficulty and stability. The techniques presented in this paper are based either on the use of reserved and slack space or the misuse of file system structures. Those techniques based on the reserved space are highly stable, but some may be detected when the file system is analysed manually. Those based on misusing file system structures (STRING_ITEMS) appear to be the most effective techniques for data hiding in the Btrfs file system.

CRedit authorship contribution statement

Fergus Toolan: Writing – review & editing, Writing – original draft, Software, Methodology, Conceptualization. **Georgina Humphries:** Writing – review & editing, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Any Btrfs file system can be used to validate the techniques.

References

- Berghele, H., Hoelzer, D., Sthultz, M., 2008. Data hiding tactics for windows and unix file systems. *Advances in Computers* 74, 1–17.
- Bhat, W.A., Wani, M.A., 2018. Forensic analysis of B-tree file system (btrfs). *Digit. Investig.* 27, 57–70.
- Btrfs (nd), On-disk format. Available at: <https://btrfs.readthedocs.io/en/latest/dev/On-disk-format.html> (Accessed March 2026a).

- Btrfs (nd), Checksumming. Available at: <https://btrfs.readthedocs.io/en/latest/Checksumming.html> (Accessed April 2026b).
- Btrfs (nd), Status. Available at: <https://btrfs.readthedocs.io/en/latest/Status.html> (Accessed April 2026c).
- Carrier, B., 2005. *File System Forensic Analysis*. Addison-Wesley Professional.
- Conlan, K., Baggili, I., Breiting, F., 2016. Anti-forensics: furthering digital forensic science through a new extended, granular taxonomy. *Digit. Investig.* 18, S66–S75.
- Fedora Project, 2020. Btrfs. Available at: <https://fedoraproject.org/wiki/Btrfs> (Accessed April 2026).
- Göbel, T., Baier, H., 2018. Anti-forensics in ext4: on secrecy and usability of timestamp-based data hiding. *Digit. Investig.* 24, S111–S120.
- Göbel, T., Baier, H., 10–12 Sep 2019. Fishy-a framework for implementing filesystem-based data hiding techniques, in: *digital forensics and cyber crime: 10th*. In: *International EAI Conference, ICDIF2C 2018*. Springer, New Orleans, LA, USA, pp. 23–42.
- Göbel, T., Baier, H., Türr, J., 2024. Generating usable and assessable datasets containing anti-forensic traces at the filesystem level. In: *IFIP International Conference on Digital Forensics*. Springer Nature Switzerland, Cham, pp. 225–246.
- González Arias, R., Bermejo Higuera, J., Rainer Granados, J.J., Bermejo Higuera, J.R., Sicilia Montalvo, J.A., 2024. Systematic review: anti-forensic computer techniques. *Appl. Sci.* 12, 5302.
- Gül, M., Kugu, E., 2017. A survey on anti-forensics techniques. In: *International Artificial Intelligence and Data Processing Symposium (IDAP)*. IEEE, pp. 1–6.
- Juch, A., 2014. *Btrfs Filesystem Forensics*. (Doctoral Dissertation, Technische Universität Wien).
- openSUSE, 2025. Sdb: btrfs. Available at: <https://en.opensuse.org/SDB:BTRFS> (Accessed April 2026).
- Rodeh, O., Bacik, J., Mason, C., 2013. BTRFS: the Linux B-tree filesystem. *ACM Transactions on Storage (TOS)* 9 (3), 1–32.
- Rogers, M., 15 Sep 2005. Anti-Forensics. Presented at Lockheed Martin, San Diego, California.
- Schwietert, A., Hilgert, J.N., 2025. Data hiding in file systems: current state, novel methods, and a standardized corpus. *Forensic Sci. Int. Digit. Investig.* 54, 301984.
- Toolan, F., 2025. *File System Forensics*. Wiley, Hoboken, NJ.
- Toolan, F., Humphries, G., 2025a. Data hiding in the XFS file system. *Forensic Sci. Int. Digit. Investig.* 52, 301884.
- Toolan, F., Humphries, G., 2025b. Data hiding in symbolic link slack space. *Forensic Sci. Int. Digit. Investig.* 53, 301919.
- Wani, M.A., Bhat, W.A., Dehghantaha, A., 2020. An analysis of anti-forensic capabilities of B-tree file system (btrfs). *Australian Journal of Forensic Sciences* 52 (4), 371–386.

Author biography

Prof. Fergus Toolan is a member of the Department of Investigation and Prosecution at the Norwegian Police University College. His research interests include file system forensics, the evaluation of digital forensic tools and the application of Information Retrieval and Machine Learning to Digital Forensics. He has published extensively in these areas. Prof. Toolan holds a master's and a PhD in Computer Science and a master's in Training and Education.

Dr. Georgina Humphries is a Dosent (Professor) at the Norwegian Police University College. Her research interests focus on digital forensics with a particular interest in education and training in the area. Dr. Humphries holds a degree in forensic computing and a PhD in the area of digital forensics education and has published extensively in the field.